Puneet Kumar 23010102S Cloud Computing

github: https://github.com/puneetkumar-2005/DistillationCommandParser

What I learnt:

A small student model can learn a narrow task from teacher by training on its outputs and may outperform the teacher in terms of accuracy and efficiency in that particular task.

Sequence level distillation: training on teacher's generated text. Only this was possible here, not logit based distillation (matching teacher's per token probabilities) as in my training using ollama, only text was generated.

Pipeline of deployed ML service: object storage using seaweed FS, parallel labelling with sharded workers, containerising fine tuned models, running on kubernetes with resource limits and setting networking rules.

Untrained models scored 95% JSON validity but performed bad at full frame which shows real understanding.

student V2 reached 45% full frame as compared to 0% of previous untrained models, beating the teacher's 44% with only 70M parameters as compared to 3.8B teacher.

CICD: make up command rebuild the whole system and quality gate catches bad model before it is deployed.

Challenges: Labelling 3000 commands on CPU was taking 14–16 hours at around 2–4 token/second. So to do the task on time, I moved the teacher to my laptop's GPU for labelling and training and then moved back to CPU only for service, load, latency measurements with the trained models.

First rolling update test failed with HTTP errors because the traffic was being routed to terminated pods. It was fixed by adding 15 seconds sleep before stopping and 45 second terminationGracePeriod which ran without errors.

During Makefile command, HPA spun down new pod before it became ready, So I changed the wait to check deployment's available condition.

Github release on website don't allow uploading large files >25 MB. so doing it in CLI remains the only way.

Observations: Untrained model: 0% full frame command passing.
Same model trained V1: 39%
trained V2: 45%
Phi 3.5 B (teacher): 44%

Service is cheap to run at ~ 0.81 per 100 requests at measured throughput as model is small and the

replicas are kept at 1GB RAM and 1 CPU each.

HPA Scaled to 2 replicas under load and back to 1 when idle.

What I could have better:

I could deploy the best among the trained models V_2, V_1 and other experimentally trained models.

Manual port forwarding in 2nd terminal can be automated.

Result summary:

storage and labelling: 2435 of 3000 total labelled outputs from both stands were kept.

model quality: V2 beats tanker with 44% vs 45%. in full frame correctness.

Latency and throughput: P50 : 1200 ms
P95 : 3000 ms

throughput : 2.71 req/sec

Resource utilization: peak 474.2 MB of 1GB

CPU is the bottleneck, not memory.

Reliability and recovery: cold start is probe delay dominated.

Rollouts worked with 0% errors.

Security and performance: credentials are only taken from secret files.

cost: ₹0.81 per 1000 requests

1 replica/hour = 3.5 / 0.45 = 3.95

request/hr = throughput × 1hr
= 2.71 × 3600 sec

$$\text{cost} = \frac{2 \times 3.95}{2.71 \times 3600} \times 1000$$
$$= 0.81 \text{ per 1000 requests.}$$